

LCLS RF Station Anomaly Candidate Dataset

The RF station anomaly dataset, located here, is composed of several files. For both AMPL and AMM, there are three files: “klys_anom_dset_{type}.h5”, “candidates_{type}.csv”, and “labels_{type}.csv”.

The HDF5 file has two top level HDF5 groups, one for *candidates* and one for *samples*, corresponding to the anomaly candidates and random samples of the beam operation respectively. Each top-level group is composed of a number of subgroups corresponding to the individual examples. The subgroups are named according to the end time of the anomaly candidate (or randomly sampled window); the time is stored as nanoseconds since epoch. Each subgroup contains two datasets called *health* and *bpm*. Each dataset is written as a 2D array of data, where the rows correspond to individual measurements and the columns correspond to the signal name. The timestamps of the rows are stored as the attribute *index* on the dataset; the values denote the number of nanoseconds past epoch. The column names are stored as the attribute *columns* on the dataset. The subgroups within *candidates* additionally have a group attribute called *klys*, which lists the RF station associated with the candidate.

The *health* dataset stores the RF diagnostic data of the given type over a period of 3.5 minutes (which corresponds to the look-back duration of the algorithm defined in ?? for AMPL). It will have shape $(n, 82)$ where n is the number of measurements and 82 is the number of RF stations. Because the RF station diagnostic data is not beam synchronous, the reporting timestamps are not aligned with one another nor the beam, meaning n can vary from dataset to dataset. In particular, it can be assumed that the AMM and AMPL diagnostic data can be reported with a delay up to roughly 5 seconds. All of the NaN values correspond to no new data for that RF station at that timestamp, and the actual value can be assumed to be equal to the last known value. If there is no good last value, then the RF station was not in use during that period of time.

The *bpm* dataset stores the BPM data over a period of roughly 17 seconds (which corresponds to the look-back duration of the algorithm defined in ??). It will have shape $(n, 8)$ where n is the number of pulses and 8 is the number of signals (2 signals for each of the 4 dispersive BPMs of interest). The number of pulses n can vary due to the rate recording and pulse-kicking behaviors mentioned in ??. This BPM data was additionally modified from the original raw data by several transforms. First, if the BPM’s TMIT value was below $1e8$, there was not enough beam to record a reliable value of the dispersion, so the

raw data merely repeats the last known value. This often erroneously repeats a “normal” value, so we artificially set the dispersive reading to 100 (well outside its feasible range) when the minimum TMIT condition is violated. Second, we apply a dispersive scaling to each BPM to place it on the order of beam energy loss. The scalings are as follows: (BPMS:LTUH:250, 0.125), (BPMS:LTUH:450, -0.125), (BPMS:DMPH:502, 0.964), and (BPMS:DMPH:693, 0.469).

The candidates CSV file lists all of the candidates and has the following columns:

1. start: Earliest time anomaly could have occurred
2. end: Latest time anomaly could have occurred
3. klys: Associated RF station
4. source: Reporting diagnostic signal (AMM or AMPL)
5. corroborate: Can take several different values, depending on the what the other diagnostic signal (AMM if considering AMPL candidates and vice versa) reports
 - (a) Missing: other diagnostic signal did not flag this candidate
 - (b) Reinforced: other diagnostic signal flagged the candidate and supports the attribution
 - (c) Disagree: other diagnostic signal flagged the candidate but disagrees on the RF station attribution
6. corr_anomaly_list: Lists the associated RF stations and the reporting diagnostic source

The labels CSV file gives the hand-labels for the candidates. Like the candidates file, it has start and end columns. It has two additional columns:

1. is_anom: Boolean indicating whether the candidate is an anomaly
2. anom_type: Either “f” for *Fault*, “s” for *Sustained anomaly*, or blank for no anomaly